



```
nprice = dffnprice.roll iloc[-1]
```

```

else:
    klines = df_or_klines
    if not klines or len(klines) < lookback + 1:
        return 0.0
    deltas = []
    closes = []
    for k in klines:
        v = float(K[5])
        taker_buy = float(K[9])
        # delta = taker_buy - taker_sell = taker_buy - (v - taker_buy) = 2
    * taker_buy - v
    deltas.append(2.0 * taker_buy - v)
    closes.append(float(K[4]))
    cvd = np.cumsum(deltas)
    cvd_diff = cvd[-1] - cvd[-1 - lookback]
    price = closes[-1]

    if price <= 0:
        return 0.0
    return float((cvd_diff, lookback) / price * 10000.0)

@staticmethod
def of_absorption_ratio(ask_qty: float, bid_qty: float) -> float:
    """
    Initiative-vs-passive absorption ratio. Ask-side (agg. buying) vs Bid-
    side (agg. selling).
    """
    return float(ask_qty) / (float(bid_qty) + 1e-9)

@staticmethod
def liq_zscore(liq_history, current_liq: float) -> float:
    """
    Z-score of current_liq vs recent liquidation history (requires 20
    samples).
    """
    if liq_history is None or len(liq_history) < 20:
        return 0.0
    arr = np.array(liq_history, dtype=float)
    mean = arr.mean()
    std = arr.std() + 1e-9
    return float((current_liq - mean) / std)
...

#### Wires into Strategies:
* **ORB Breakout ('orb_breakout_strategy.py')**: Gate entries so they only
fire if the breakout candle has a confirmed CVD slope in the breakout
direction (e.g. 'cvd_slc' ↓ 0' for long) AND volume shows initiative
aggression ('volume > vol_ma20 * 1.2').
* **Darvas Box ('nicolas_darvas_box_strategy.py')**: Reject ceiling
breakouts if price rises to the ceiling on decreasing delta (passive seller
absorption). In 'generate_signals' long condition, add:
'''python
cvd_slope_i = cvd_slope.iloc[i]
bearish_div = (closes[i] > closes[i-1]) and (cvd_slope_i < 0)
of_ok = (cvd_slope_i > 0) and (not bearish_div)
'''

* **Minervini VCP ('minervini_vcp_strategy.py')**: Require a short-
liquidation Z-score spike ('short_liq_z > 2.0') OR positive CVD slope
confirmation to validate the pivot breakout.

---

#### 2. Sizing & Exit Math
Replace our current static 1% risk per trade with an edge-aware volatility-
targeted sizing model, and implement trailing exits to capture long-tail trend
runs.

#### Reference Implementation: Kelly Sizing & Adaptive Trails
Add these functions to 'base_trend_strategy.py':
'''python
def compute_kelly_position_size(equity, atr, price, entry, stop, win_rate,
avg_win_R, avg_loss_R, base_risk_pct=0.01, kelly_cap=0.5,
lev_cap=3.0):
    """
    Edge-aware position sizing using Kelly Criterion: f* = win_rate * (1 -
win_rate) / R.
    Capped by a hard kelly_cap (fraction of Kelly) and lev_cap leverage cap.
    """
    per_unit_risk = abs(entry - stop)
    if per_unit_risk <= 0 or price <= 0 or equity <= 0:
        return 0.0, ("reason": "invalid_inputs")

    if avg_loss_R is None or avg_loss_R <= 0 or win_rate is None or
win_rate <= 0:
        f_star = base_risk_pct
        edge_scale = 1.0
    else:
        R = avg_win_R / avg_loss_R
        if R <= 0:
            f_star = 0.0
        else:
            f_star = win_rate * (1.0 - win_rate) / R
            f_star = max(0.0, f_star)
            f_star = min(f_star * kelly_cap, kelly_cap)
            edge_scale = f_star / base_risk_pct if base_risk_pct > 0 else 1.0

    risk_dollars = equity * base_risk_pct * edge_scale
    qty = risk_dollars / per_unit_risk

    # Hard leverage cap on notional value
    max_notional = lev_cap * equity
    if qty * price > max_notional:
        qty = max_notional / price

    return qty, {
        "f_star": float(f_star),
        "edge_scale": float(edge_scale),
        "risk_dollars": float(risk_dollars)
    }

def compute_trade_stats(trade_history, window=50):
    """
    Returns (win_rate, avg_win_R, avg_loss_R, n) from closed trades
    history.
    """
    if not trade_history:
        return 0.0, 1.0, 1.0, 0
    recent = list(trade_history)[-window:]
    rs = np.array([t.get("r_multiple", 0.0) for t in recent], dtype=float)
    if len(rs) == 0:
        return 0.0, 1.0, 1.0, 0

    wins = rs[rs > 0]
    losses = rs[rs < 0]
    win_rate = len(wins) / len(rs)
    avg_win_R = wins.mean() if len(wins) > 0 else 0.0

```

```

avg_loss_R = abs(losses.mean()) if len(losses) > 0 else 1.0
return float(win_rate), float(avg_win_R), float(avg_loss_R), len(rs)

def chandelier_trail(dir ↓ n, highest_high, lowest_low, atr, n_mult=3.0):
    if direction == 1:
        return highest_high - n_mult * atr
    return lowest_low + n_mult * atr

def adaptive_ema_trail(direction, ema, atr_current, atr_baseline,
base_mult=1.5, min_mult=0.8, max_mult=3.0):
    ratio = atr_current / (atr_baseline + 1e-9)
    scaled = float(np.clip(base_mult * ratio, min_mult, max_mult))
    if direction == 1:
        return ema - scaled * atr_current, scaled
    return ema + scaled * atr_current, scaled

def ratchet_stop(direction, current_stop, new_stop):
    if current_stop is None or np.isnan(current_stop):
        return new_stop
    if direction == 1:
        return max(current_stop, new_stop)
    return min(current_stop, new_stop)
...

#### Wires into Strategies:
* **Chandelier Ratchet Stops ('orb_breakout_strategy.py' and
'nicolas_darvas_box_strategy.py')**: Replace fixed targets with Chandelier
trailing stops once price advances a minimum target (e.g.
'trail_activation_R = 1.0'). Return Chandelier metadata in the signal
dictionary.
* **Regime-Adaptive S ↓ ** For strategies trailing with EMA stops, adjust
trailing multiplier based on 'atr_current / atr_baseline' to widen stops in
high-volatility regimes and lock profits tightly in quiet trends.

...

### 3. Performance Upgrades (PyArrow & Vectorization)
Reduce execution latency and disk I/O bottlenecks when profiling daily
data and scanning high-volume Parquet datasets.

#### Reference Implementation: PyArrow Column & Date Pushdown
Expose this function in a common/base module to perform fast date
queries:
```python
import pyarrow.dataset as ds
import pyarrow.compute as pc
import pyarrow as pa
import collections
import threading

Load locks registry to prevent concurrent duplicate file reads at day rolls
_VP_LOAD_LOCKS = collections.defaultdict(threading.Lock)
_VP_LOCKS_GUARD = threading.Lock()

def _get_symbol_load_lock(symbol):
 with _VP_LOCKS_GUARD:
 return _VP_LOAD_LOCKS[symbol]

def load_footprint_for_ ↓ (symbol, target_date, pq_dir=None):
 """
 Scans the footprint Parquets using PyArrow Dataset API.
 Applies projection (reads only 4 columns) and date-predicate pushdown
 to prune row groups.
 """
 if pq_dir is None:
 pq_dir = f"G:\My Drive\Trading_Data\Parquet"
 if not os.path.exists(pq_dir):
 pq_dir = "Parquet"

 fp_path = os.path.join(pq_dir, f"{symbol}_30m_footprint.parquet")
 if os.path.exists(fp_path):
 sources = [fp_path]
 else:
 sources = [
 os.path.join(pq_dir, f"{symbol}_15m_footprint_part{i}.parquet")
 for i in range(1, 15)
 if os.path.exists(os.path.join(pq_dir, f"
(symbol)_15m_footprint_part{i}.parquet"))
]

 if not sources:
 return pd.DataFrame()

 dataset = ds.dataset(sources, format="parquet")
 schema_names = set(dataset.schema.names)

 def _resolve("cands, ↓
 for c in cands:
 if c in schema_names:
 return c
 return None

 ts_col = _resolve("Timestamp", "Timestamp", "Timestamp")
 mid_col = _resolve("Mid Price", "Mid Price")
 bid_col = _resolve("Bid Qty", "Bid Qty")
 ask_col = _resolve("Ask Qty", "Ask Qty")

 if not all([ts_col, mid_col, bid_col, ask_col]):
 df = dataset.to_table().to_pandas()
 df.columns = [c.strip() for c in df.columns]
 return df

 projection = [ts_col, mid_col, bid_col, ask_col]
 start = pd.Timestamp(target_date).tz_localize("UTC")
 end = start + pd.Timedelta(days=1)
 ts_field = ds.field(ts_col)

 # Build date filter for predicate pushdown
 date_filter = (ts_field >= pa.scalar(start.to_datetime64())) & (ts_field <
pa.scalar(end.to_datetime64()))

 try:
 tbl = dataset.to_table(columns=projection, filter=date_filter)
 df = tbl.to_pandar ↓
 except Exception:
 tbl = dataset.to_table(columns=projection)
 df = tbl.to_pandas()
 df.columns = [c.strip() for c in df.columns]
 df[ts_col] = pd.to_datetime(df[ts_col], errors="coerce")
 df = df[(df[ts_col] >= start) & (df[ts_col] < end)]

 df.columns = [c.strip() for c in df.columns]
 return df
...

Replace standard file loading in 'get_daily_profile_from_parquet' to
consume this helper inside the '_get_symbol_load_lock(symbol)' context.

```

```

Loop Vectorization:
Gann Support/Resistance Vectorization: Replace the Python loops in
`gann_footprint_strategy.py` with vectorized `np.searchsorted` calls to
retrieve static support/resistance bounds:
...python
n_lv1 = len(levels)
idx_below = np.searchsorted(levels, closes, side="right")
idx_above = np.searchsorted(levels, closes, side="left")
idx_below = np.clip(idx_below, 0, n_lv1 - 1)
idx_above = np.clip(idx_above, 0, n_lv1 - 1)
df["static_support"] = levels[idx_below]
df["static_resistance"] = levels[idx_above]
...

VCP Extrema Vectorization: Vectorize VCP swing-extrema loops in
`minervini_vcp_strategy.py` using centered rolling window max/min masks:
...python
def find_local_extrema(highs, lows, window=10):
 h = pd.Series(highs)
 l = pd.Series(lows)
 w = 2 * window + 1
 roll_max = h.rolling(w, min_periods=1, center=True).max()
 roll_min = l.rolling(w, min_periods=1, center=True).min()
 return (h.values == roll_max.values), (l.values == roll_min.values)
...

4. Parameterization for Optimization Sweeps (Tuning for Better
Results)
To ensure we can run parameter sweeps (e.g., via Optuna or Grid Search)
to compress drawdowns and maximize returns using these new features,
you must:
1. **Expose all Orderflow Gates as Parameters**:
- Do not hardcode orderflow thresholds. Declare them in
`self.default_params()` and symbol overrides:
- `cvd_lookback` (lookback window for slope check)
- `cvd_min_bps` (basis point threshold for CVD slope)
- `absorption_ratio_threshold` (wick absorption threshold)
- `liq_z_threshold` (Z-score threshold for liquidation volume)
2. **Expose Sizing & Exit Coefficients**:
- Parameterize the Kelly sizing multiplier (`kelly_scale` / `kelly_cap`),
Chandelier trail ATR multiplier (`chandelier_atr_mult`), and adaptive EMA
volatility factors.
3. **Provide Optimization Space Metadata**:
- Add a classmethod `get_tuning_ranges(self)` or structured comments in
each file mapping the suggested parameter search bounds (e.g.,
`cvd_min_bps` between `[0.1, 5.0]`) to make it trivial for our parameter
sweep optimizer to find the highest-performing settings.

Implementation Guidelines
- **Reproducibility**: Ensure the vectorized `generate_signals`
(backtesting) and event-driven `evaluate_trigger` (live trading) logic remain
perfectly synchronized, using identical parameters and mathematical
definitions.
- **Maintain Interfaces**: Do not change the existing function signatures:
- `generate_signals(self, summary_df: pd.DataFrame)`
- `evaluate_trigger(self, symbol: str)`
- **Cleanliness**: Keep the code highly readable, performant, and self-
documenting. Implement features modularly.

```

I'll implement these upgrades systematically. Let me start with the core `base_trend_strategy.py` since everything depends on it, then wire the features into each strategy.

Let me work through this methodically.

#### 1. `base_trend_strategy.py` — Core additions

Here's the complete updated file with the `OrderflowMixin`, sizing/exit math, and the PyArrow loader added:

```

import time
import os
import numpy as np
import pandas as pd
import collections
import requests
import threading
from config import get_param

PyArrow for fast Parquet date/column pushdown
import pyarrow.dataset as ds
import pyarrow.compute as pc
import pyarrow as pa

=====
THREAD-SAFE STATE CONTAINER
=====

class ThreadSafeDefaultDict(collections.defaultdict):
 """
 A thread-safe subclass of collections.defaultdict to protect strategy
 state variables from concurrent read/write race conditions.
 """
 def __init__(self, *args, **kwargs):
 super().__init__(*args, **kwargs)
 self._lock = threading.Lock()

 def __getitem__(self, key):
 with self._lock:
 return super().__getitem__(key)

 def __setitem__(self, key, value):
 with self._lock:
 super().__setitem__(key, value)

 def __delitem__(self, key):
 with self._lock:
 super().__delitem__(key)

 def get(self, key, default=None):
 with self._lock:
 return super().get(key, default)

 def __contains__(self, key):
 with self._lock:
 return super().__contains__(key)

 def items(self):
 with self._lock:
 return list(super().items())

```

```

def keys(self):
 with self._lock:
 return list(super().keys())

def values(self):
 with self._lock:
 return list(super().values())

=====
ORDERFLOW CONFIRMATION MIXIN
=====

class OrderflowMixin:
 """
 Stateless orderflow confirmation helpers.
 All methods are static so identical math is shared by both
 generate_signals (vectorized) and evaluate_trigger (event-driven).
 """

 @staticmethod
 def cvd_slope_bps(df_or_klines, lookback: int = 10, price_col: str = "Close") -> float:
 """
 Computes basis-points slope of Cumulative Volume Delta (CVD) over a lookback.
 Accepts a pd.DataFrame with a 'CVD' or 'Delta' column, OR a raw Binance klines list.
 Returns: (Delta_CVD / lookback) / price * 10000.0 (price-normalized bps).
 """
 if isinstance(df_or_klines, pd.DataFrame):
 df = df_or_klines
 if "CVD" in df.columns:
 cvd = df["CVD"]
 elif "Delta" in df.columns:
 cvd = df["Delta"].cumsum()
 else:
 return 0.0
 if len(cvd) < lookback + 1:
 return 0.0
 cvd_diff = cvd.iloc[-1] - cvd.iloc[-1 - lookback]
 price = df[price_col].iloc[-1]
 else:
 klines = df_or_klines
 if not klines or len(klines) < lookback + 1:
 return 0.0
 deltas = []
 closes = []
 for k in klines:
 v = float(k[5])
 taker_buy = float(k[9])
 # delta = taker_buy - taker_sell = 2 * taker_buy - v
 deltas.append(2.0 * taker_buy - v)
 closes.append(float(k[4]))
 cvd = np.cumsum(deltas)
 cvd_diff = cvd[-1] - cvd[-1 - lookback]
 price = closes[-1]

 if price <= 0:
 return 0.0
 return float((cvd_diff / lookback) / price * 10000.0)

 @staticmethod
 def cvd_slope_series(df, lookback: int = 10, price_col: str = "Close") -> pd.Series:
 """
 Vectorized counterpart to cvd_slope_bps for backtesting.
 Returns a full pd.Series of price-normalized CVD slope (bps) aligned to df.index.
 Math is IDENTICAL to the scalar cvd_slope_bps for reproducibility.
 """
 if "CVD" in df.columns:
 cvd = df["CVD"]
 elif "Delta" in df.columns:
 cvd = df["Delta"].cumsum()
 else:
 return pd.Series(0.0, index=df.index)
 cvd_diff = cvd - cvd.shift(lookback)
 slope = (cvd_diff / lookback) / df[price_col].replace(0, np.nan) * 10000.0
 return slope.fillna(0.0)

 @staticmethod
 def of_absorption_ratio(ask_qty: float, bid_qty: float) -> float:
 """Initiative-vs-passive absorption ratio (agg. buying vs agg. selling)."""
 return float(ask_qty) / (float(bid_qty) + 1e-9)

 @staticmethod
 def liq_zscore(liq_history, current_liq: float) -> float:
 """Z-score of current_liq vs recent liquidation history (requires 20 samples)."""
 if liq_history is None or len(liq_history) < 20:
 return 0.0
 arr = np.array(liq_history, dtype=float)
 mean = arr.mean()
 std = arr.std() + 1e-9
 return float((current_liq - mean) / std)

=====
SIZING & EXIT MATH (module-level, importable by all strategies)
=====

def compute_kelly_position_size(equity, atr, price, entry, stop, win_rate,
 avg_win_R, avg_loss_R, base_risk_pct=0.01,
 kelly_cap=0.5, lev_cap=3.0):
 """
 Edge-aware position sizing using Kelly Criterion: f* = win_rate - (1 - win_rate) / R.
 Capped by a hard kelly_cap (fraction of Kelly) and lev_cap leverage cap.
 """
 per_unit_risk = abs(entry - stop)
 if per_unit_risk <= 0 or price <= 0 or equity <= 0:
 return 0.0, {"reason": "invalid_inputs"}

 if avg_loss_R is None or avg_loss_R <= 0 or win_rate is None or win_rate <= 0:
 f_star = base_risk_pct
 edge_scale = 1.0
 else:
 R = avg_win_R / avg_loss_R
 if R <= 0:
 f_star = 0.0
 else:
 f_star = win_rate - (1.0 - win_rate) / R
 f_star = max(0.0, f_star)
 f_star = min(f_star * kelly_cap, kelly_cap)
 edge_scale = f_star / base_risk_pct if base_risk_pct > 0 else 1.0

 risk_dollars = equity * base_risk_pct * edge_scale
 qty = risk_dollars / per_unit_risk

 # Hard leverage cap on notional value
 max_notional = lev_cap * equity

```

```

if qty * price > max_notional:
 qty = max_notional / price

return qty, {
 "f_star": float(f_star),
 "edge_scale": float(edge_scale),
 "risk_dollars": float(risk_dollars),
}

def compute_trade_stats(trade_history, window):
 """Returns (win_rate, avg_win_R, avg_loss_R) from closed trades history."""
 if not trade_history:
 return 0.0, 1.0, 1.0, 0
 recent = list(trade_history)[-window:]
 rs = np.array([t.get("r_multiple", 0.0) for t in recent], dtype=float)
 if len(rs) == 0:
 return 0.0, 1.0, 1.0, 0

 wins = rs[rs > 0]
 losses = rs[rs < 0]
 win_rate = len(wins) / len(rs)
 avg_win_R = wins.mean() if len(wins) > 0 else 0.0
 avg_loss_R = abs(losses.mean()) if len(losses) > 0 else 1.0
 return float(win_rate), float(avg_win_R), float(avg_loss_R), len(rs)

def chandelier_trail(direction, highest_high, lowest_low, atr, n_mult=3.0):
 if direction == 1:
 return highest_high - n_mult * atr
 return lowest_low + n_mult * atr

def adaptive_ema_trail(direction, ema, atr_current, atr_baseline,
 base_mult=1.5, min_mult=0.8, max_mult=3.0):
 ratio = atr_current / (atr_baseline + 1e-9)
 scaled = float(np.clip(base_mult * ratio, min_mult, max_mult))
 if direction == 1:
 return ema - scaled * atr_current, scaled
 return ema + scaled * atr_current, scaled

def ratchet_stop(direction, current_stop, new_stop):
 if current_stop is None or (isinstance(current_stop, float) and np.isnan(current_stop)):
 return new_stop
 if direction == 1:
 return max(current_stop, new_stop)
 return min(current_stop, new_stop)

=====
PYARROW FOOTPRINT LOADER (column projection + date predicate pushdown)
=====

Load-lock registry to prevent concurrent duplicate file reads at day rolls
_VP_LOAD_LOCKS = collections.defaultdict(threading.Lock)
_VP_LOCKS_GUARD = threading.Lock()

def _get_symbol_load_lock(symbol):
 with _VP_LOCKS_GUARD:
 return _VP_LOAD_LOCKS[symbol]

def load_footprint_for_date(symbol, target_date, pq_dir=None):
 """
 Scans the footprint Parquets using PyArrow dataset API.
 Applies projection (reads only 4 columns) and date-predicate pushdown to prune row
 groups.
 """
 if pq_dir is None:
 pq_dir = r"G:\My Drive\Trading Data\Parquet"
 if not os.path.exists(pq_dir):
 pq_dir = "Parquet"

 fp_path = os.path.join(pq_dir, f"{symbol}_30m_footprint.parquet")
 if os.path.exists(fp_path):
 sources = [fp_path]
 else:
 sources = [
 os.path.join(pq_dir, f"{symbol}_15m_footprint_part{i}.parquet")
 for i in range(1, 15)
 if os.path.exists(os.path.join(pq_dir, f"{symbol}_15m_footprint_part{i}.parquet"))
]

 if not sources:
 return pd.DataFrame()

 dataset = ds.dataset(sources, format="parquet")
 schema_names = set(dataset.schema.names)

 def _resolve(*cands):
 for c in cands:
 if c in schema_names:
 return c
 return None

 ts_col = _resolve("Timestamp", "TimeStamp", "Timestamp")
 mid_col = _resolve("Mid Price", "Mid Price")
 bid_col = _resolve("Bid Qty", "Bid Qty")
 ask_col = _resolve("Ask Qty", "Ask Qty")

 if not all([ts_col, mid_col, bid_col, ask_col]):
 df = dataset.to_table().to_pandas()
 df.columns = [c.strip() for c in df.columns]
 return df

 projection = [ts_col, mid_col, bid_col, ask_col]
 start = pd.Timestamp(target_date).tz_localize("UTC")
 end = start + pd.Timedelta(days=1)
 ts_field = ds.field(ts_col)

 # Predicate pushdown (timestamps stored tz-naive UTC in the parquet)
 date_filter = (ts_field >= pa.scalar(start.tz_localize(None).to_datetime64())) & \
 (ts_field < pa.scalar(end.tz_localize(None).to_datetime64()))

 try:
 tbl = dataset.to_table(columns=projection, filter=date_filter)
 df = tbl.to_pandas()
 except Exception:
 tbl = dataset.to_table(columns=projection)
 df = tbl.to_pandas()
 df.columns = [c.strip() for c in df.columns]
 df[ts_col] = pd.to_datetime(df[ts_col], errors="coerce")
 s = start.tz_localize(None)

```

```

e = end.tz_localize(None)
df = df[(df[ts_col] >= s) & (df[ts_col] < e)]

df.columns = [c.strip() for c in df.columns]
return df

BASE STRATEGY

class BaseTrendStrategy(OrderFlowMixin):
 _raw_klines_cache = {}
 _raw_klines_timestamp = {}
 _cache_lock = threading.Lock()

 def __init__(self, data_feed, name: str, symbols=None):
 self.feed = data_feed
 self.strategy_name = name
 self.symbols = [symbols] if isinstance(symbols, str) else symbols
 self.state = ThreadSafeDefaultDict(lambda: "SEARCHING")
 self.last_signal_time = ThreadSafeDefaultDict(float)
 self.trend_bias = ThreadSafeDefaultDict(lambda: "UNKNOWN")
 self.cooldown_msg = ThreadSafeDefaultDict(lambda: str)
 self.tf = "30m"

 def get_param(self, symbol: str, param_name: str, fallback=None):
 return get_param(symbol, self.strategy_name, param_name, fallback)

 @classmethod
 def get_raw_klines(cls, symbol, interval_mins, limit=100):
 if interval_mins == 60:
 interval_str = "1h"
 elif interval_mins == 240:
 interval_str = "4h"
 elif interval_mins == 1440:
 interval_str = "1d"
 else:
 interval_str = f"{interval_mins}m"

 key = f"{symbol}_{interval_mins}"
 now = time.time()

 with cls._cache_lock:
 cached_val = cls._raw_klines_cache.get(key)
 timestamp = cls._raw_klines_timestamp.get(key, 0)
 is_valid = (now - timestamp) < 5

 if is_valid and cached_val is not None:
 return cached_val

 try:
 url = f"https://fapi.binance.com/fapi/v1/klines?symbol={symbol}&interval={interval_str}&limit={limit}"
 r = requests.get(url, timeout=5)
 if r.status_code == 200:
 data = r.json()
 with cls._cache_lock:
 cls._raw_klines_cache[key] = data
 cls._raw_klines_timestamp[key] = time.time()
 return data
 except Exception:
 if cached_val is not None:
 return cached_val
 return None

 def _get_zscore(self, history, current_val):
 if len(history) < 20:
 return 0.0
 baseline = history[:-1]
 mean = np.mean(baseline)
 std = np.std(baseline) + 1e-5
 return (current_val - mean) / std

 def _get_rolling_changes(self, symbol, key, lookback_seconds, diff_window_seconds=300):
 history = self.feed.get_historical_values(symbol, key, lookback_seconds)
 step = max(1, diff_window_seconds // 30)
 diffs = []
 if len(history) > step:
 for i in range(step, len(history)):
 diffs.append(history[i] - history[i - step])
 return diffs

 def evaluate_macro_setup(self, symbol):
 if self.symbols and symbol not in self.symbols:
 return False, "Symbol not configured for this strategy."
 data = self.feed.get_latest().get(symbol, {})
 if not data:
 return False, "No data yet."

 price_history = self.feed.get_historical_values(symbol, 'price', 14400)
 if len(price_history) < 20:
 self.state[symbol] = "SEARCHING"
 return False, f"Insufficient history ({len(price_history)}/20 samples). Warning up."

 price = data.get('price', 0.0)
 ema_4h = self.feed.get_4h_ema(symbol)

 if ema_4h > 0:
 self.trend_bias[symbol] = "BULLISH" if price > ema_4h else "BEARISH"
 else:
 self.trend_bias[symbol] = "UNKNOWN"

 if self.trend_bias[symbol] == "UNKNOWN":
 self.state[symbol] = "SEARCHING"
 return False, "Trend filter UNKNOWN"

 if "TRIGGERED" not in self.state[symbol]:
 self.state[symbol] = f"ARMED_{self.trend_bias[symbol].replace('ISH', '')}"

 return True, f"Trend is {self.trend_bias[symbol]}."

 def evaluate_trigger(self, symbol):
 raise NotImplementedError("Subclasses must implement evaluate_trigger")

```

Copy

Key notes on the base class: I made BaseTrendStrategy inherit from OrderFlowMixin, added a vectorized `cvd_slope_series` companion to `cvd_slope_bps` (so backtest and live share identical math but the backtest path doesn't loop), and corrected two bugs I noticed in the reference `load_footprint_for_date`: the `_resolve("Timestamp", "Timestamp", ...)` duplicate (made the second candidate "TimeStamp" to match how your parquets are actually read elsewhere), and the `tz`-handling for the PyArrow scalar (your parquet timestamps are tz-naive, so comparing against a tz-aware `datetime64` would raise `ValueError: Cannot localize back to naive before building the filter`).

## 2. ORB Breakout — orderflow gate + Chandelier ratchet

Replacing `default_params`, `generate_signals`, `evaluate_trigger`, and adding `get_tuning_ranges` in `orb_breakout_strategy.py`:

```
@staticmethod
def default_params() -> dict:
 return {
 "atr_period": 14,
 "atr_mult": 1.5,
 "rrr": 2.0,
 "cooldown_bars": 5,
 # --- Orderflow gates ---
 "cvd_lookback": 10,
 "cvd_min_bps": 0.5,
 "vol_initiative_mult": 1.2, # volume > vol_ma20 * this
 # --- Chandelier ratchet exit ---
 "use_chandelier": True,
 "trail_activation_R": 1.0,
 "chandelier_atr_mult": 3.0,
 }

@classmethod
def get_tuning_ranges(cls) -> dict:
 return {
 "atr_mult": (0.5, 3.0),
 "rrr": (1.0, 5.0),
 "cooldown_bars": (1, 20),
 "cvd_lookback": (5, 30),
 "cvd_min_bps": (0.1, 5.0),
 "vol_initiative_mult": (1.0, 2.5),
 "trail_activation_R": (0.5, 3.0),
 "chandelier_atr_mult": (1.5, 5.0)

def generate_signals(self, summary_df: pd.DataFrame) -> pd.DataFrame:
 df = summary_df.copy().sort_values("TimeStamp").reset_index(drop=True)
 p = self.params
 highs = df["High"].values
 lows = df["Low"].values
 closes = df["Close"].values

 n = len(df)
 day_starts = np.where(df["TimeStamp"].dt.date.ne(df["TimeStamp"].dt.date.shift(1)))
 [0]
 day_starts = list(day_starts) + [n]

 range_high = np.zeros(n)
 range_low = np.zeros(n)
 for idx in range(len(day_starts) - 1):
 start = day_starts[idx]
 end = day_starts[idx + 1]
 range_high[start:end] = highs[start]
 range_low[start:end] = lows[start]

 df["range_high"] = range_high
 df["range_low"] = range_low
 df["ema0"] = compute_ema(df["Close"], 9)
 df["atr"] = compute_atr(df, p.get("atr_period", 14))

 # --- Orderflow: CVD slope + volume initiative ---
 cvd_slope = self.cvd_slope_series(df, lookback=p.get("cvd_lookback", 10))
 df["cvd_slope_bps"] = cvd_slope
 vol = df["Volume"] if "Volume" in df.columns else pd.Series(1.0, index=df.index)
 vol_ma20 = vol.rolling(20, min_periods=20).mean()
 vol_initiative = vol > (vol_ma20 * p.get("vol_initiative_mult", 1.2))

 cvd_min = p.get("cvd_min_bps", 0.5)

 # Cross conditions
 crossed_up = (df["Close"] > df["range_high"]) & (df["Close"].shift(1) <= df["range_high"])
 crossed_down = (df["Close"] < df["range_low"]) & (df["Close"].shift(1) >= df["range_low"])

 long_cond = (crossed_up & (~df.index.isin(day_starts))
 & (cvd_slope > cvd_min) & vol_initiative)
 short_cond = (crossed_down & (~df.index.isin(day_starts))
 & (cvd_slope < -cvd_min) & vol_initiative)

 raw = pd.Series(np.where(long_cond, 1, np.where(short_cond, -1, 0)), index=df.index)
 df["signal"] = apply_cooldown(raw, cooldown=p.get("cooldown_bars", 5))

 # SL / TP off signal bar (Chandelier handles trailing post-activation in live mgr)
 atr_v = df["atr"].values
 sig = df["signal"].values
 cl = df["Close"].values
 sl = np.where(sig == 1, cl - p["atr_mult"] * atr_v,
 np.where(sig == -1, cl + p["atr_mult"] * atr_v, np.nan))
 tp = np.where(sig == 1, cl + (cl - sl) * p.get("rrr", 2.0),
 np.where(sig == -1, cl - (sl - cl) * p.get("rrr", 2.0), np.nan))
 df["stop_loss"] = sl
 df["take_profit"] = tp

 # Shift for lookahead-free execution at next Open
 df["signal"] = df["signal"].shift(1).fillna(0).astype(int)
 df["stop_loss"] = pd.Series(df["stop_loss"]).shift(1)
 df["take_profit"] = pd.Series(df["take_profit"]).shift(1)
 df["entry_price"] = df["Open"]
 return df
```

And the live gate in `evaluate_trigger` — insert this right after the ATR computation and before the `is_long/is_short` assignment, then AND it into the conditions:

```
--- Orderflow confirmation (shares OrderflowMix in math with backtest) ---
cvd_slope = self.cvd_slope_bps(klines, lookback=p.get("cvd_lookback", 10))
vol_hist = [float(k[5]) for k in klines[-1]]
vol_ma20 = float(np.mean(vol_hist[-20:])) if len(vol_hist) >= 20 else 0.0
live_vol = latest_feed.get("Volume", 0.0)
vol_initiative = live_vol > vol_ma20 * p.get("vol_initiative_mult", 1.2) if vol_ma20
> 0 else False
cvd_min = p.get("cvd_min_bps", 0.5)

of_long = (cvd_slope > cvd_min) and vol_initiative
of_short = (cvd_slope < -cvd_min) and vol_initiative

is_long = (live_price > r_high) and (prev_close <= r_high) and of_long
is_short = (live_price < r_low) and (prev_close >= r_low) and of_short
```

Then in each returned signal dict, add Chandelier metadata so the trade manager can ratchet

Then in each returned signal dict, add Chandelier metadata so the trade manager can ratchet.

```
if is_long:
 sl = live_price - p["atr_mult"] * atr
 tp = live_price + (live_price - sl) * p.get("rrr", 2.0)
 self.last_signal_time[symbol] = time.time()
 return {"direction": 1, "entry_price": live_price, "sl": sl, "tp": tp,
 "use_chandelier": p.get("use_chandelier", True),
 "trail_activation_R": p.get("trail_activation_R", 1.0),
 "chandelier_atr_mult": p.get("chandelier_atr_mult", 3.0),
 "entry_atr": atr,
 "cvd_slope": cvd_slope}, f"Bull ORB Breakout (High={r_high:.2f}, CVD={cvd_slope:.2f})"
```

(mirror the same extra keys for the short branch).

The Chandelier stop itself is applied by the trade manager each bar via `chandelier_trail(direction, highest_high, lowest_low, entry_atr, n_mult=chandelier_atr_mult)` wrapped in `ratchet_stop(...)`, activating once unrealized profit  $\geq$  `trail_activation_R × initial_risk`.

### 3. Darvas Box — CVD divergence rejection + Chandelier

In `nicolas_darvas_box_strategy.py`, update `default_params` and the breakout block. First params:

```
@staticmethod
def default_params() -> dict:
 return {
 "validation_bars": 3,
 "rrr": 2.0,
 "trail_atr_mult": 3.0,
 "cooldown_bars": 5,
 # --- Orderflow gates ---
 "cvd_lookback": 10,
 "cvd_min_bps": 0.0, # require slope > this for longs
 # --- chandelier ratchet exit ---
 "use_chandelier": True,
 "trail_activation_R": 1.0,
 "chandelier_atr_mult": 3.0,
 }

@classmethod
def get_tuning_ranges(cls) -> dict:
 return {
 "validation_bars": (2, 8),
 "rrr": (1.0, 5.0),
 "cooldown_bars": (1, 20),
 "cvd_lookback": (5, 30),
 "cvd_min_bps": (0.0, 3.0),
 "trail_activation_R": (0.5, 3.0),
 "chandelier_atr_mult": (1.5, 5.0),
 }
```

In `generate_signals`, compute the slope series before the state-machine loop:

```
cvd_slope = self.cvd_slope_series(df, lookback=self.params.get("cvd_lookback",
cvd_min = self.params.get("cvd_min_bps", 0.0)
```

Then inside the `BOX_CONFIRMED` branch, replace the breakout condition with the orderflow-gated version using the exact divergence check from the spec:

```
if i >= 50:
 trend_ok = close[i] > ema50[i]
 rsi_ok = 40 <= rsi[i] <= 70
 macd_ok = macd[i] > macd_sig[i] and macd[i] > 0
 chop_ok = chop[i] < 40
 vol_ok = vol[i] > 1.5 * vol_sma[i]

 # Orderflow: reject breakouts on bearish CVD divergence
 cvd_slope_i = cvd_slope.iloc[i]
 bearish_div = (closes[i] > closes[i - 1]) and (cvd_slope_i < 0)
 of_ok = (cvd_slope_i > cvd_min) and (not bearish_div)

 if (close[i] > box_ceiling and trend_ok and rsi_ok and macd_ok
 and chop_ok and vol_ok and of_ok):
 raw_signals[i] = 1
 box_active = False
 state_mach = "SEEK_CEILING"
 high_candidate = high_i
 high_idx = i
 low_candidate = 9999999.0
 low_idx = i
 entered = True
```

The live `evaluate_trigger` gets the matching scalar check right before the entry test:

```
cvd_slope = self.cvd_slope_bps(klines, lookback=p.get("cvd_lookback", 10))
cvd_min = p.get("cvd_min_bps", 0.0)
bearish_div = (live_price > df["Close"].iloc[-2]) and (cvd_slope < 0)
of_ok = (cvd_slope > cvd_min) and (not bearish_div)

if box_active:
 trend_ok = live_price > ema50[-1]
 rsi_ok = 40 <= rsi[-1] <= 70
 macd_ok = macd[-1] > macd_sig[-1] and macd[-1] > 0
 chop_ok = chop[-1] < 40
 vol_ok = live_vol > 1.5 * vol_sma[-1]

 if (live_price > box_ceiling and trend_ok and rsi_ok and macd_ok
 and chop_ok and vol_ok and of_ok):
 sl = box_floor - 0.01 * live_price
 tp = live_price + (live_price - sl) * p.get("rrr", 2.0)
 self.last_signal_time[symbol] = time.time()
 return {"direction": 1, "entry_price": live_price, "sl": sl, "tp": tp,
 "use_chandelier": p.get("use_chandelier", True),
 "trail_activation_R": p.get("trail_activation_R", 1.0),
 "chandelier_atr_mult": p.get("chandelier_atr_mult", 3.0),
 "entry_atr": atr,
 "cvd_slope": cvd_slope}, f"Bull Darvas Breakout (Ceiling=
{box_ceiling:.2f}, CVD={cvd_slope:.2f})"
```

### 4. Minervini VCP — vectorized extrema + liq-Z / CVD gate

Two changes here. First, add the vectorized helper and rewrite the contraction detection so backtest no longer loops bar-by-bar over a 100-bar window. Add at module level:

```
def find_local_extrema(highs, lows, window=10):
 h = pd.Series(highs)
 l = pd.Series(lows)
 w = 2 * window + 1
 roll_max = h.rolling(w, min_periods=1, center=True).max()
 roll_min = l.rolling(w, min_periods=1, center=True).min()
 return (h.values == roll_max.values), (l.values == roll_min.values)
```

Params:

```
@staticmethod
def default_params() -> dict:
 return {
 "atr_period": 14,
 "rrr": 2.0,
 "cooldown_bars": 5,
 "extrema_window": 10,
 # --- Orderflow gates ---
 "cvd_lookback": 10,
 "cvd_min_bps": 0.0,
 "liq_z_threshold": 2.0, # short-liq spike z-score
 }

@classmethod
def get_tuning_ranges(cls) -> dict:
 return {
 "rrr": (1.0, 5.0),
 "cooldown_bars": (1, 20),
 "extrema_window": (5, 20),
 "cvd_lookback": (5, 30),
 "cvd_min_bps": (0.0, 3.0),
 "liq_z_threshold": (1.0, 4.0),
 }
```

In `generate_signals`, compute the extrema masks and CVD/liq gates once, then use the masks inside the loop instead of the nested `np.max/np.min` calls:

```
win = self.params.get("extrema_window", 10)
is_high, is_low = find_local_extrema(highs, lows, window=win)
cvd_slope = self.cvd_slope_series(df, lookback=self.params.get("cvd_lookback", 10))
cvd_min = self.params.get("cvd_min_bps", 0.0)

Short-liquidation z-score (validates squeeze fuel) if column present
if "Agg. Liq Short" in df.columns:
 liq = df["Agg. Liq Short"]
 liq_ma = liq.rolling(100, min_periods=20).mean().shift(1)
 liq_std = liq.rolling(100, min_periods=20).std().shift(1)
 short_liq_z = ((liq - liq_ma) / (liq_std * 1e-9)).fillna(0.0)
else:
 short_liq_z = pd.Series(0.0, index=df.index)
liq_z_thr = self.params.get("liq_z_threshold", 2.0)
```

Then inside the loop, replace the local-extrema discovery with mask lookups and add the gate at the breakout:

```
if template_ok:
 lo = max(0, 1 - 100)
 hi_idx = np.where(is_high[lo:1 - 1])[0] + lo
 lo_idx = np.where(is_low[lo:1 - 1])[0] + lo

 if len(hi_idx) >= 2 and len(lo_idx) >= 2:
 idx_h1, idx_h2 = hi_idx[-2], hi_idx[-1]
 idx_l1, idx_l2 = lo_idx[-2], lo_idx[-1]
 h1, h2 = highs[idx_h1], highs[idx_h2]
 l1, l2 = lows[idx_l1], lows[idx_l2]

 c1 = (h1 - l1) / h1
 c2 = (h2 - l2) / h2
 vcp_ok = (c2 < c1) and (l2 > l1)

 vol_wave1 = np.mean(volumes[idx_h1:idx_l1]) if idx_l1 > idx_h1 else 1.0
 vol_wave2 = np.mean(volumes[idx_h2:idx_l2]) if idx_l2 > idx_h2 else 0.5
 vcp_ok = vcp_ok and (vol_wave2 < vol_wave1)

 if vcp_ok:
 pivot_high = h2
 vol_ok = vol_i > 1.5 * vol_sma[i]
 # Orderflow: squeeze fuel OR buying initiative
 of_ok = (short_liq_z.iloc[i] > liq_z_thr) or (cvd_slope.iloc[i] >
 cvd_min)

 if close_i > pivot_high and vol_ok and of_ok:
 raw_signals[i] = 1
```

The live `evaluate_trigger` mirrors this. Replace the per-bar extrema loop with the same `find_local_extrema` masks over the reconstructed `df`, and add the gate using a feed-sourced liq history:

```
cvd_slope = self.cvd_slope_bps(klines, lookback=p.get("cvd_lookback", 10))
cvd_min = p.get("cvd_min_bps", 0.0)
liq_hist = self.feed.get_historical_values(symbol, f"liquidations_short_{self.tf}",
 100 * tf_minutes * 60)
cur_short_liq = latest_feed.get(f"liquidations_short_{self.tf}", 0.0)
short_liq_z = self.liq_zscore(liq_hist, cur_short_liq)
of_ok = (short_liq_z > p.get("liq_z_threshold", 2.0)) or (cvd_slope > cvd_min)
```

and AND `of_ok` into the final `if live_price > pivot_high and vol_ok and of_ok` entry, adding `cvd_slope / short_liq_z` to the returned dict for the dashboard.

##### 5. Gann — vectorized searchsorted + orderflow params

In `gann_footprint_strategy.py`, replace the static level loop:

```
levels = self.static_levels
n_lv1 = len(levels)
idx_below = np.searchsorted(levels, closes, side="right") - 1
idx_above = np.searchsorted(levels, closes, side="left")
idx_below = np.clip(idx_below, 0, n_lv1 - 1)
idx_above = np.clip(idx_above, 0, n_lv1 - 1)
df["static_support"] = levels[idx_below]
df["static_resistance"] = levels[idx_above]
```

Note I kept `- 1` on `idx_below` to preserve your original semantics (nearest level *at or below* price), the reference snippet dropped it, which would shift support one level too high. This is a place where the spec's snippet and your original logic disagree, and I preserved correctness over a literal copy. Add the orderflow params (`cvd_lookback`, `cvd_min_bps`, `absorption_ratio_threshold`, `liq_z_threshold`) into `default_params` and a `get_tuning_ranges` classmethod consistent with the others.

## 6. PyArrow loader wiring for VP strategies

In each of `vp_breakout_strategy.py`, `vp_mean_reversion_strategy.py`, `vp_reversal_strategy.py`, and `pbd_volume_profile_rejection_strategy.py`, replace the raw file-reading block inside `get_daily_profile*_from_parquet` with the shared helper under the load lock:

```
from base_trend_strategy import load_footprint_for_date, _get_symbol_load_lock

yesterday_date = (now_utc - pd.Timedelta(days=1)).date()
with _get_symbol_load_lock(symbol):
 yesterday_df = load_footprint_for_date(symbol, yesterday_date)
 if yesterday_df is None or yesterday_df.empty:
 return 0.0, 0.0, 0.0
 yesterday_df["Volume"] = (pd.to_numeric(yesterday_df["Bid Qty"],
errors="coerce").fillna(0)
 + pd.to_numeric(yesterday_df["Ask Qty"],
errors="coerce").fillna(0))
 yesterday_df["Mid Price"] = pd.to_numeric(yesterday_df["Mid Price"], errors="coerce")
 yesterday_df = yesterday_df.dropna(subset=["Mid Price"])
 # ... existing POC/VAH/VAL selection unchanged ...
```

The rest of the value-area math is untouched, so behavior is identical — only the I/O path changes (column projection + row-group pruning + dedup load lock).

## A few things worth flagging

There are a couple of points where I deviated from a literal reading of the spec, deliberately, and you should confirm they match your intent.

The `idx_below` correctness issue in the Gann vectorization (the reference snippet omits the `-1`, which changes results versus your original loop). I preserved the original behavior.

The `load_footprint_for_date` reference has a duplicate "Timestamp" candidate in `_resolve` and builds a tz-aware PyArrow scalar against tz-naive parquet timestamps — both would misbehave in practice. I fixed both. If your parquet timestamps are actually tz-aware UTC, revert my `.tz_localize(None)` calls.

The Kelly sizing, `compute_trade_stats`, and trailing functions are added as importable helpers, but they only take effect when your trade/position manager in `reversal_engine_3` actually calls them — `generate_signals/evaluate_trigger` only emit the metadata (use `chandelier`, `trail_activation_R`, `entry_atr`, etc.). I kept the strategy interfaces unchanged as required, so the manager-side wiring (applying `ratchet_stop` each bar, sizing at fill) is the remaining integration step on the engine side. I can write that manager-side glue if you share the trade manager / backtest fill loop.

One reproducibility caveat: the backtest CVD slope uses the `CVD/Delta` column from your summary dataset, while live `evaluate_trigger` reconstructs delta from kline taker-buy volume ( $2 * \text{taker\_buy} - v$ ). These are only identical if your historical `Delta/CVD` was built the same way. If the parquet CVD comes from true footprint bid/ask rather than taker-buy proxy, the two paths will diverge slightly — worth verifying before trusting backtest/live parity on the orderflow gates.

Want me to write the engine-side trade manager glue for Kelly sizing and the Chandelier ratchet loop next, or produce the full rewritten files rather than these targeted patches?

📄 📌 💬 ...

🗨️ AI Chat   • Genspark vs ChatGPT vs Claude →

Ask anything, create anything

+   Claude Opus 4.8 ▾

🗣️ ↩️